

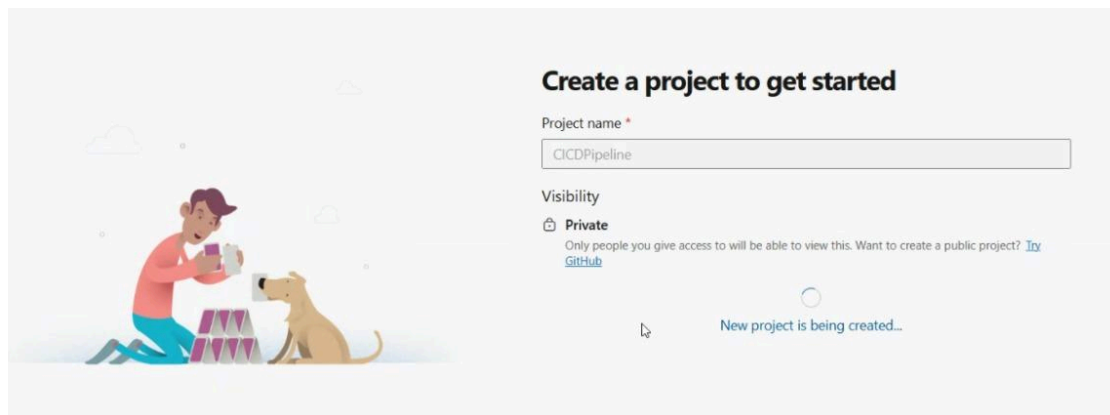
Pipeline creation in Azure DevOps to apply CI/CD

Introduction

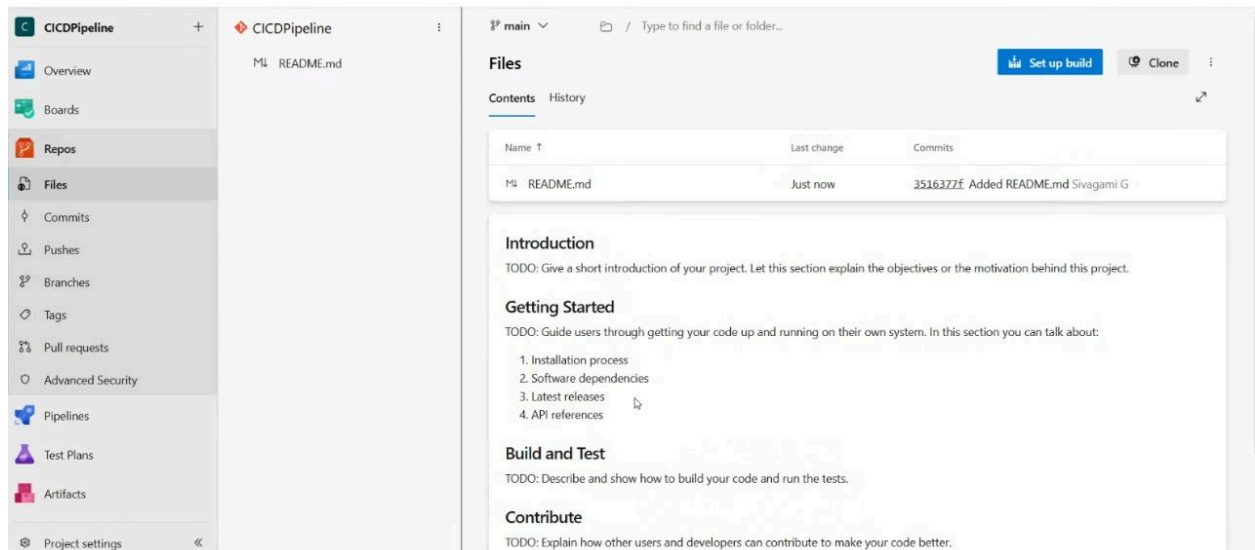
CI/CD (Continuous Integration and Continuous Deployment) is a process that automates code building, testing, and deployment. In this project, we implemented a simple CI/CD pipeline in Azure DevOps by creating a repository with a Python script, setting up a YAML pipeline, and configuring a self-hosted agent pool. The pipeline was executed successfully, running the Python script and demonstrating the automation of code execution through CI/CD.

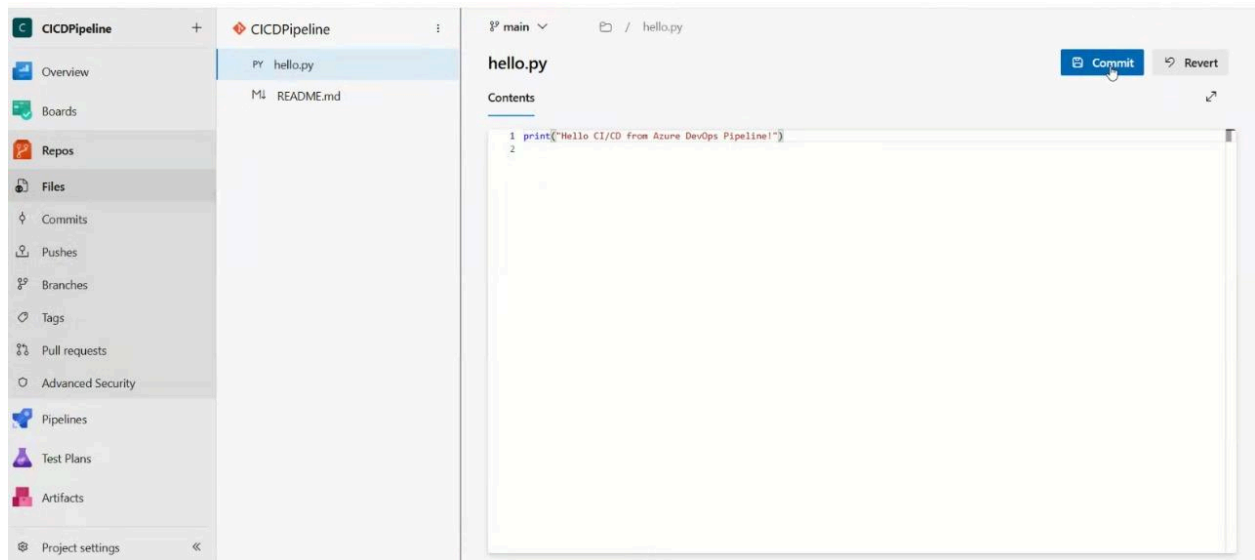
Project Setup

Creating Azure DevOps organization and project.



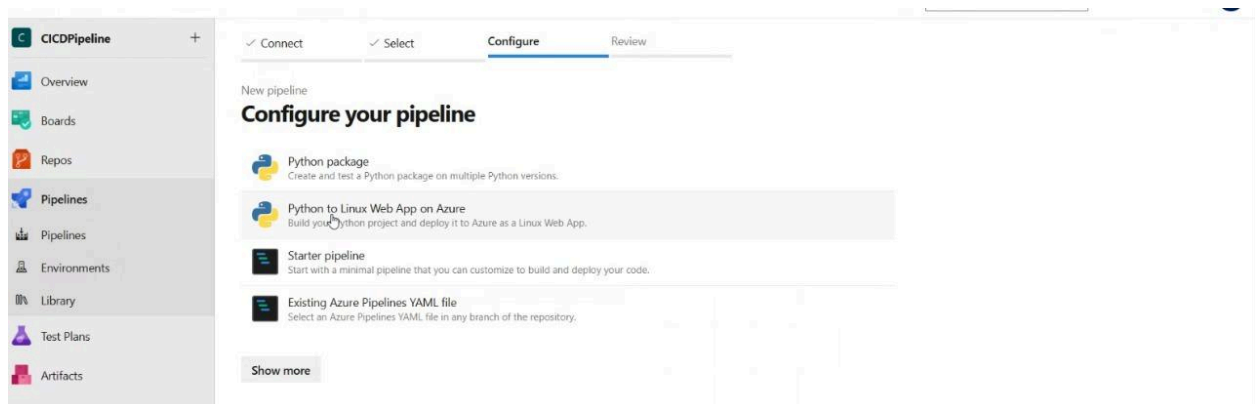
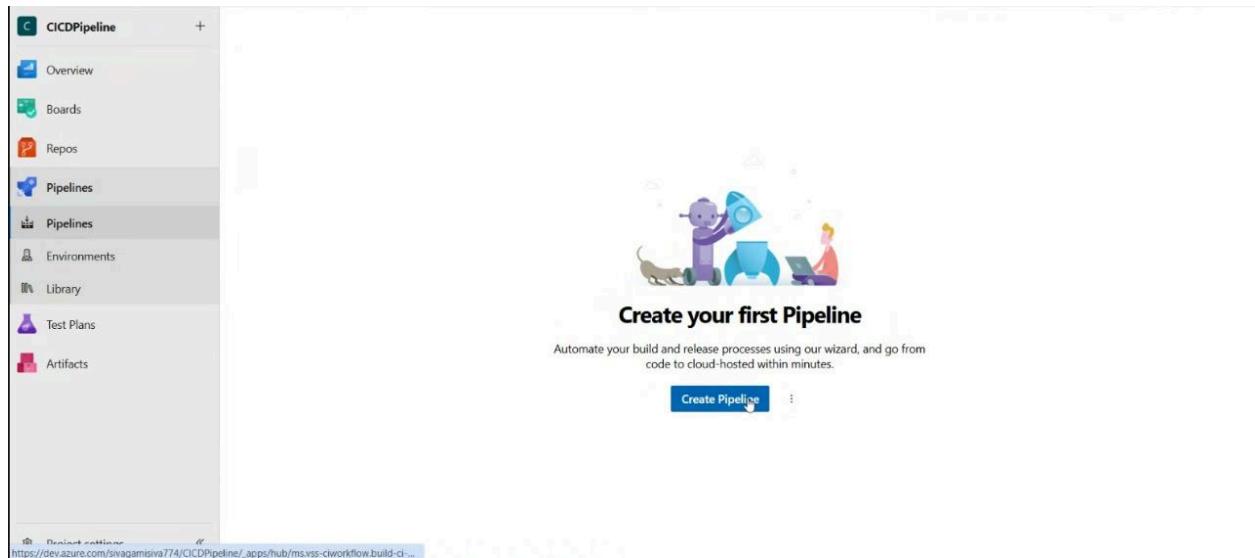
Initializing repository with README and Python file.





Pipeline Setup

Creating the pipeline in Azure DevOps.



YAML configuration for Python script execution.

The screenshot shows the Azure DevOps web interface for configuring a new pipeline. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Library, Test Plans, and Artifacts. The main area is titled 'Review your pipeline YAML' and displays a YAML configuration for a pipeline named 'CICDPipeline / azure-pipelines.yml'. The configuration includes a trigger for the 'main' branch, a pool using 'ubuntu-latest', and two steps: one to echo 'Hello, world!' and another to echo a message about adding tasks. The 'Save and run' button is visible in the top right corner.

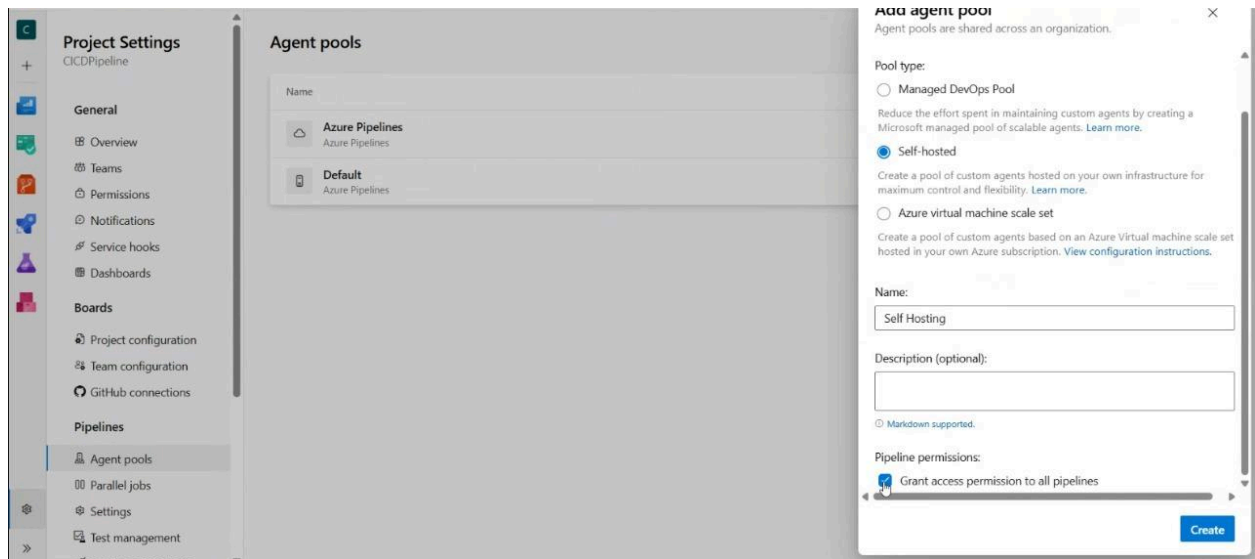
```
1 # Starter pipeline
2 # Start with a minimal pipeline that you can customize to build and deploy your code.
3 # Add steps that build, run tests, deploy, and more:
4 # https://aka.ms/yaml
5
6 trigger:
7   - main
8
9 pool:
10  vmImage: ubuntu-latest
11
12 steps:
13   - script: echo Hello, world!
14     displayName: 'Run a one-line script'
15
16   - script: |
17       echo Add other tasks to build, test, and deploy your project.
18       echo See https://aka.ms/yaml
19     displayName: 'Run a multi-line script'
20
```

This screenshot shows the 'Save and run' dialog box that appears after clicking the 'Save and run' button. It prompts the user to save the pipeline configuration to a repository. The dialog includes a 'Commit message' field with the default text 'Set up CI with Azure Pipelines', an 'Optional extended description' field, and radio buttons to choose between 'Commit directly to the main branch' (selected) and 'Create a new branch for this commit'. A 'Creating pipeline...' progress indicator is shown at the bottom right.

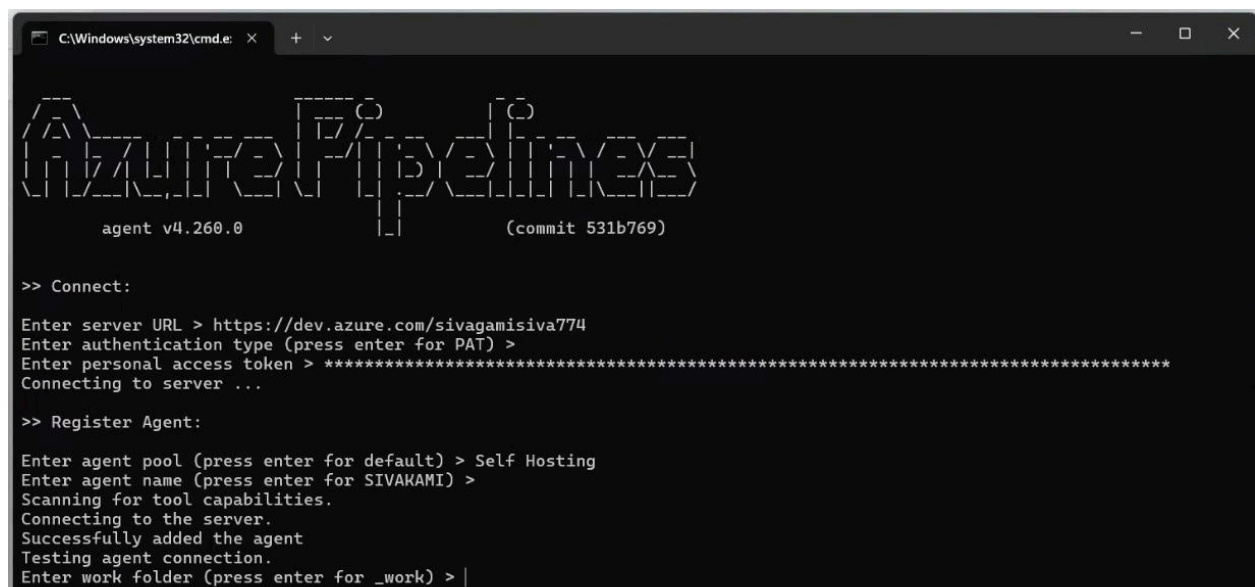
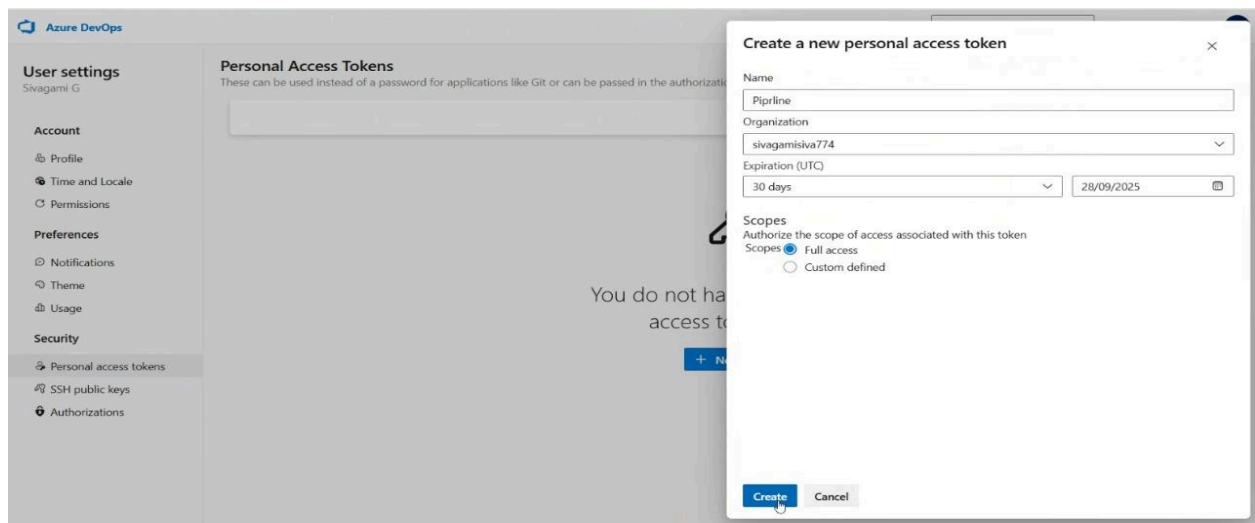
The screenshot displays the 'Run pipeline' dialog box, which allows for manually running a pipeline. It shows the selected branch/tag as 'main'. Under 'Advanced options', it indicates that there are no defined variables, the pipeline will run as configured, and it will use the latest version of all resources. There is a checkbox for 'Enable system diagnostics' and 'Cancel' and 'Run' buttons at the bottom.

Self-Hosted Agent

Setting up the self-hosted agent and agent pool.

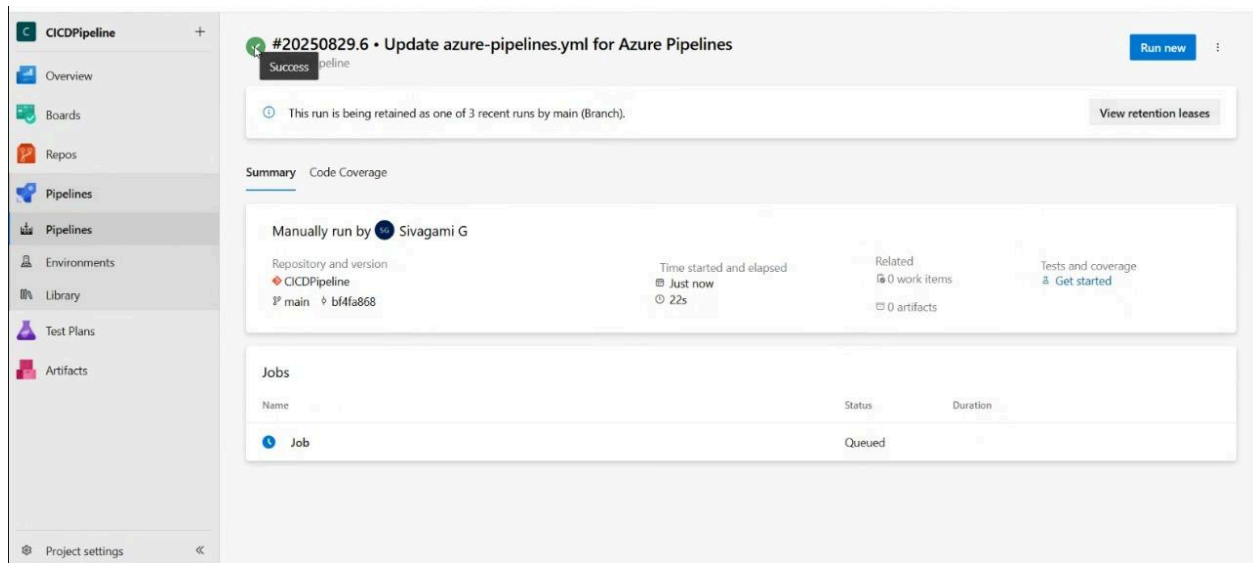


Linking the pipeline to the self-hosted agent.



Pipeline Execution

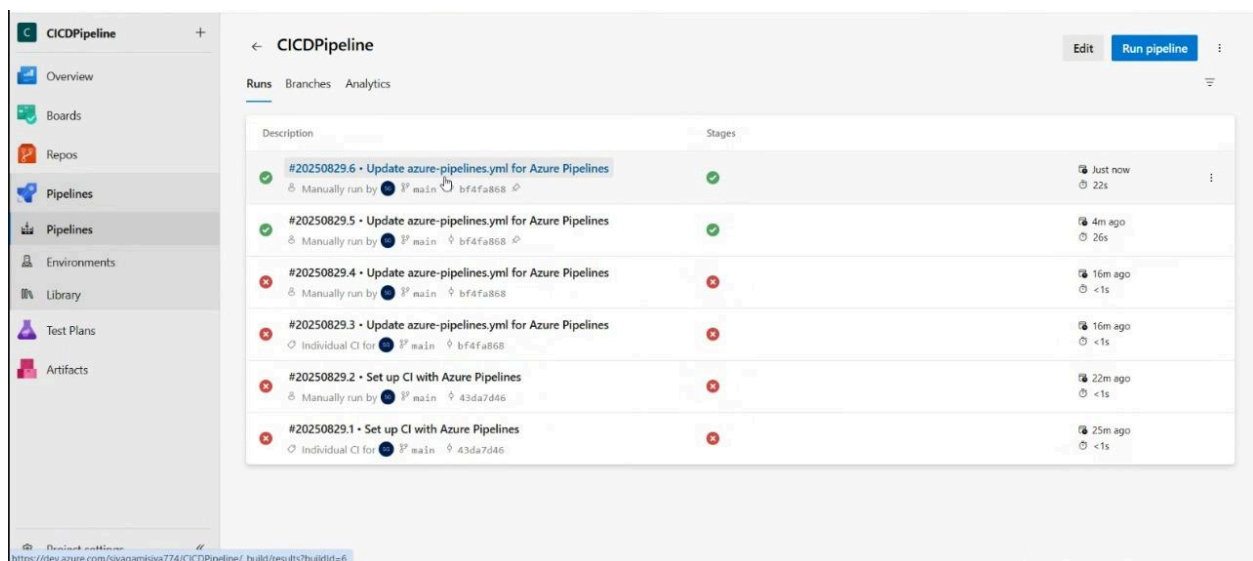
Running the pipeline.



The screenshot shows the Azure DevOps Pipelines interface for a project named 'CICDPipeline'. The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines, Environments, Library, Test Plans, and Artifacts. The main area displays the details of a pipeline run with ID #20250829.6, titled '#20250829.6 • Update azure-pipelines.yml for Azure Pipelines'. The status is 'Success'. A 'Run new' button is in the top right. Below the status, a message states: 'This run is being retained as one of 3 recent runs by main (Branch)'. A 'View retention leases' button is also present. The 'Summary' tab is active, showing 'Manually run by Sivagami G'. It lists the repository and version as 'CICDPipeline' and 'main' with commit 'bf4fa868'. It also shows 'Time started and elapsed' as 'Just now' and '22s', 'Related' as '0 work items' and '0 artifacts', and 'Tests and coverage' as 'Get started'. A 'Jobs' table is shown below with one job named 'Job' in a 'Queued' status.

Name	Status	Duration
Job	Queued	

Output/result of the pipeline run.



The screenshot shows the 'Runs' tab of the 'CICDPipeline' in Azure DevOps. It displays a list of pipeline runs with columns for Description, Stages, and a timestamp. The runs are as follows:

Description	Stages	Timestamp
#20250829.6 • Update azure-pipelines.yml for Azure Pipelines Manually run by Sivagami G main bf4fa868	Success	Just now 22s
#20250829.5 • Update azure-pipelines.yml for Azure Pipelines Manually run by Sivagami G main bf4fa868	Success	4m ago 26s
#20250829.4 • Update azure-pipelines.yml for Azure Pipelines Manually run by Sivagami G main bf4fa868	Failed	16m ago <1s
#20250829.3 • Update azure-pipelines.yml for Azure Pipelines Individual CI for Sivagami G main bf4fa868	Failed	16m ago <1s
#20250829.2 • Set up CI with Azure Pipelines Manually run by Sivagami G main 43da7d46	Failed	22m ago <1s
#20250829.1 • Set up CI with Azure Pipelines Individual CI for Sivagami G main 43da7d46	Failed	25m ago <1s

The URL at the bottom of the page is https://dev.azure.com/sivagamisiva774/CICDPipeline/_build/results?buildId=6.

Conclusion

In this project, we implemented a basic CI/CD pipeline using Azure DevOps. We began by creating an organization and project, then initialized a repository with a Python script. A pipeline was created using a YAML file, and instead of relying on Microsoft-hosted agents, we configured a self-hosted agent pool to run the pipeline. The pipeline successfully executed the Python script, proving the automation flow. Through this exercise, we learned how CI/CD works in practice, how to set up and customize pipelines, and the importance of agent configuration. It gave us a clear understanding of how even simple code changes can be automated and integrated into a professional DevOps workflow.